

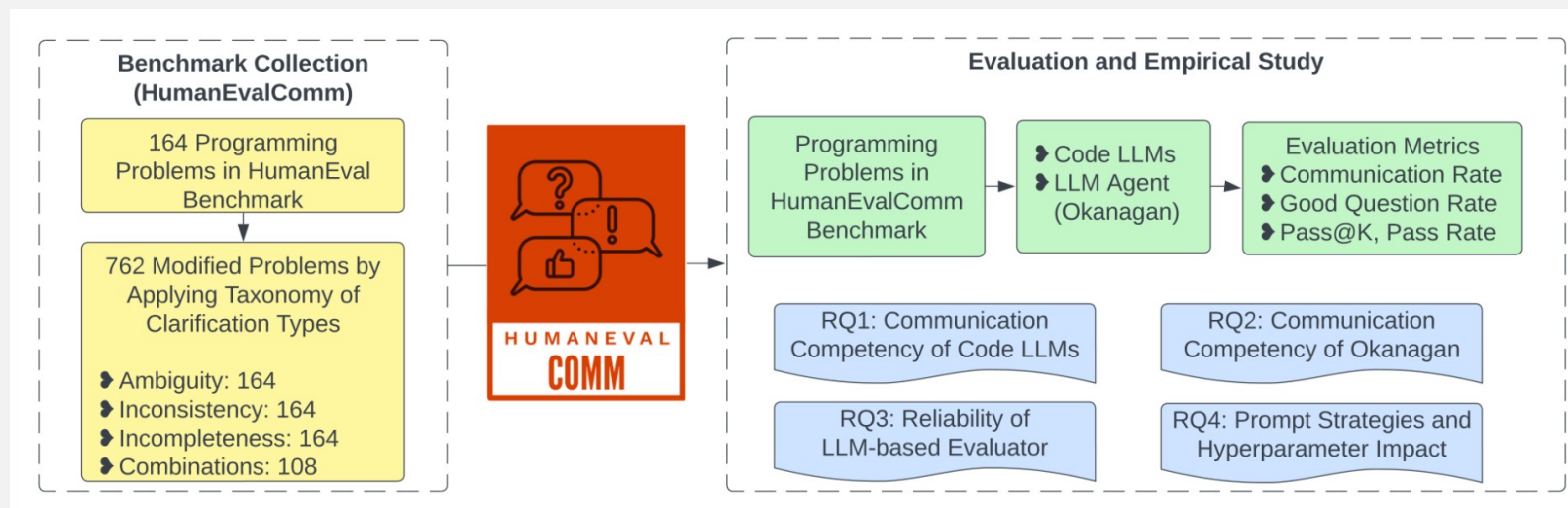
# Software Engineering with Generative AI

Week 5 · RLHF, reasoning, code tutorial  
Instructor: Jie “JW” Wu  
CS 5090 · Fall 2026  
Michigan Technological University



# What if prompt is unclear?

<https://jie-jw-wu.github.io/blog/does-ai-ask.html>



# Fine-Tuning Case Study

**Problem:** Code Models should increase the ability to ask clarifying questions for coding tasks, but how?

## Key Observation:

Only {coding question, code} pairs exist in training data, causing LLMs to generate code regardless

- Such instruction tuning methods in Code LLMs **force** the models to generate code, regardless of any issues in the problem description.

## Our Vision:

At the model level, LLMs **themselves** should have the ability to ask clarifying questions when needed

# Example #1

```
#Fixed scalar addition
def increment_list(last):
    return [x + 5 for x in lst]

#Parameterized increment
def increment_list(lst, num):
    return [x + num for x in lst]

#Position-dependent increment
def increment_list(last):
    return [x + i for i, x in enumerate(lst)]

#Type-conditional increment
def increment_list(last):
    return [x+1 if isinstance(x, int) else x \
            for x in lst]
```

Ambiguous implementations for the problem statement **“Return list with elements incremented by a number”**. Variants demonstrate alternative interpretations of the problem.

# Example #2



Q:  
from typing import List

```
def candidate(numbers: List[float], threshold: float) -> bool:
    """ Check if in given list of numbers, are any two numbers meeting a
    certain condition (e.g. closer to or larger than each other than given
    threshold).
    >>> candidate([1.0, 2.0, 3.0], 0.5)
    False
    >>> candidate([1.0, 2.8, 3.0, 4.0, 5.0, 2.0], 0.3)
    True
    """
```

w/o ClarifyCoder

```
def has_close_elements(numbers: List[float], threshold: float) ->
bool:
    """ Check if in given list of numbers, are any two numbers closer
    to each other than
    given threshold.
    >>> has_close_elements([1.0, 2.0, 3.0], 0.5)
    False
    >>> has_close_elements([1.0, 2.8, 3.0, 4.0, 5.0, 2.0], 0.3)
    True
    """
    if len(numbers) < 2:
        return False
    seen = set()
    for num in numbers:
        close_to = [n for n in numbers if abs(num - n) < threshold and
n not in seen]
        if close_to:
            seen.update(close_to)
    return len(seen) > 0
```



w/ ClarifyCoder

A: The problem description is not clear. It mentions "closer to or larger than each other than given threshold", but it does not specify what exactly is meant by "closer".

Please clarify the definition of "closer" or "larger" in the context of the problem.



# Asking Clarifying Questions All the Time?



User interaction with clarifying questions is enforced for standard coding tasks that could be completed without clarifications.

ClarifyGPT, TICODER, and ClariGen: explicitly require the user interaction module

Okanagan: asks clarifying questions only when necessary but, asks unnecessary questions, affecting pass@1



LLM-based workflows are complicated and costly than a single LLM.

ClariGen and Okanagan: need 3 calls to LLMs

ClarifyGPT: needs 4 calls



We need a balance between asking unnecessary questions and truly needed questions.

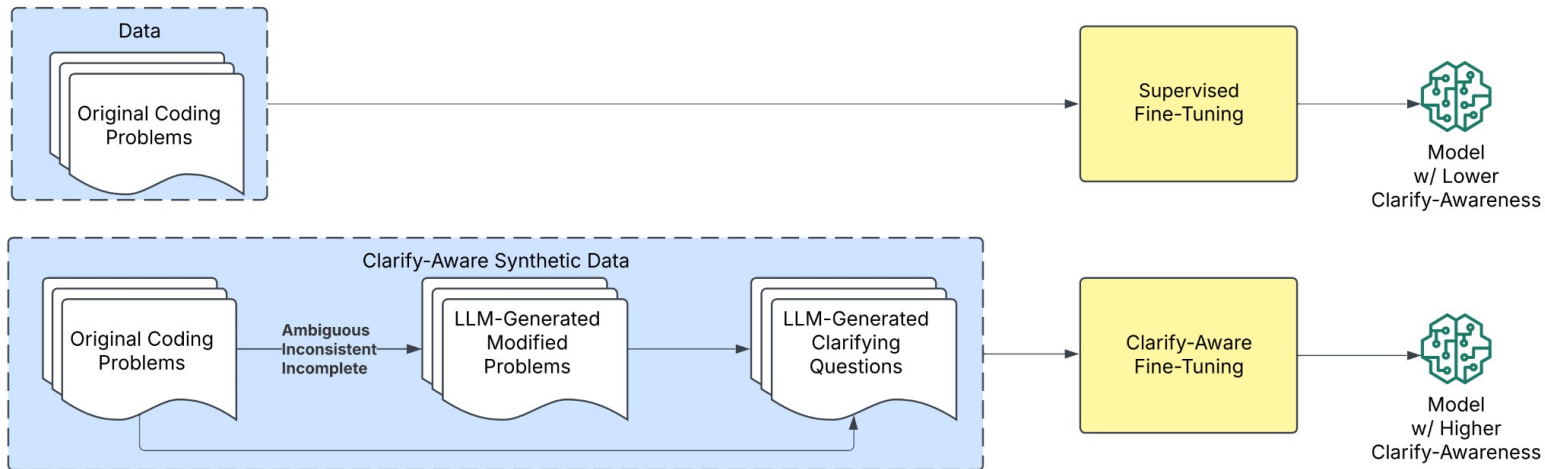
# Clarification-Aware Fine-Tuning



Goal: **alignment** for clarification-aware code generation.

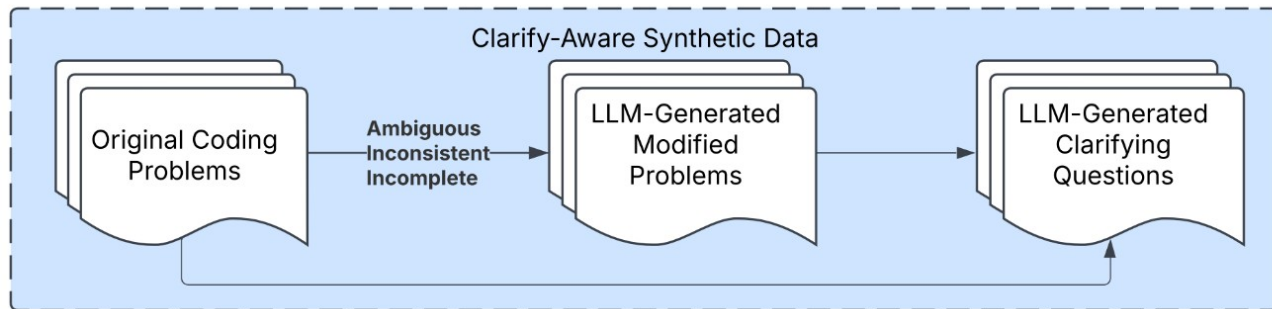
We propose **ClarifyCoder** that **fine-tunes** the Code LLMs with

- Synthetic Data Construction
- Clarification-Aware Instruction Tuning



# Clarification-Aware Data Synthesis Process

**Goal:** produce ambiguous problem statements and their corresponding clarifying questions, thereby creating a robust dataset for training ClarifyCoder.



# Clarification-Aware Data Synthesis Process

- Modify problems into the three categories
- Ask the model to generate a question to clarify the problem description
- Using APPS data, ~30 K new data are generated



## Ambiguous

- Modification: Rewrite to introduce ambiguity by creating multiple valid interpretations or leaving key details unspecified.
- Question Gen: Identify points of ambiguity and formulate clarifying questions that resolve them.



## Incomplete

- Modification: Rewrite to create incompleteness by omitting key concepts or conditions essential for solving.
- Question Gen: Identify points of incompleteness and formulate clarifying questions to resolve them.



## Inconsistent

- Modification: Rewrite to introduce inconsistency by incorporating conflicting statements.
- Question Gen: Identify points of inconsistency and formulate clarifying questions to provide a consistent interpretation.

# Clarification-Aware Fine-Tuning

We create a unified dataset  $D^{all}$  by merging  $D^{clarify}$  and  $D^{og}$  :

$$D^{all} = D^{clarify} \cup D^{og}$$

We then perform instruction tuning on  $D^{all}$  by minimizing the combined loss function  $\mathcal{L}^{all}$  :

$$\mathcal{L}^{all} = - \sum_{t=1}^{|a^{all}|} \log P(a_t^{all} | a_{<t}^{all}, p^{all})$$

Example result:

---

**Algorithm 1** Combined Instruction Tuning for Code Generation

---

**Require:** Pre-trained LLM

**Require:**  $D_{og}$ : Standard instruction tuning dataset

**Require:**  $D_{clarify}$ : Clarify-aware instruction tuning dataset

**Require:**  $r$ : Ratio  $|D_{clarify}| / (|D_{og}| + |D_{clarify}|)$

**Ensure:** Instruction-tuned LLM

- 1:  $D^{all} \leftarrow$  Combine  $D^{og}$  and  $D^{clarify}$  according to ratio  $r$  and chosen sampling strategy
  - 2: **for** each sample  $s \in D^{all}$  **do**
  - 3:     Optimize the LLM on  $s$  by minimizing  $\mathcal{L}^{all}$  (Equation 8)
  - 4: **end for**
  - 5: **return** Instruction-tuned LLM
- 

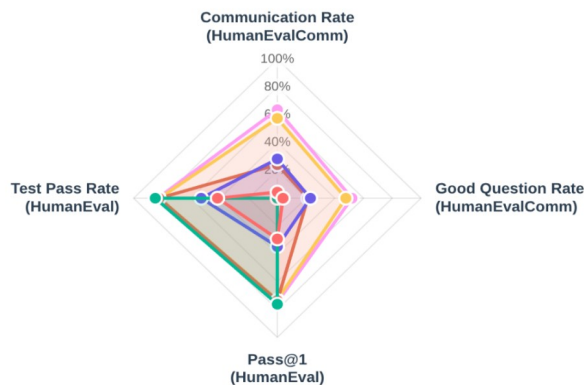
| Ambiguous Problem Statement                                                                                                                                                                              | DeepSeek Coder Response                                                                                                                                                          | ClarifyCoder Response                                                                                                                                                              |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>def candidate(l):<br/>    """<br/>    candidate takes a list of integers as an<br/>    input.<br/>    it returns True if there are two distinct<br/>    elements in the list that<br/>    """</pre> | <pre>def candidate(l):<br/>    seen = set()<br/>    for num in l:<br/>        if num in seen:<br/>            return True<br/>        seen.add(num)<br/>    return False\n</pre> | It seems like your function description is incomplete. Could you please provide more details about what the function should return if there are two distinct elements in the list? |

# Performance Analysis (1 / 2)

| Model                                           | HumanEvalComm        |                      | HumanEval            |                      |
|-------------------------------------------------|----------------------|----------------------|----------------------|----------------------|
|                                                 | Comm.<br>Rate        | Good Q.<br>Rate      | Pass@1               | Test Pass<br>Rate    |
| CodeLlama                                       | 4.38%                | 3.93%                | 29.27%               | 41.63%               |
| CodeQwen1.5 Chat                                | 0%                   | 0%                   | <b><u>76.22%</u></b> | <b><u>84.95%</u></b> |
| DeepSeek Chat                                   | <b>28.25%</b>        | <b>23.03%</b>        | 34.76%               | 53.00%               |
| DeepSeek Coder                                  | <b>24.12%</b>        | <b>21.29%</b>        | <b>73.62%</b>        | <b>82.36%</b>        |
| ClarifyCoder (w/ DS Coder)                      | <b><u>57.42%</u></b> | <b><u>47.68%</u></b> | <b>73.01%</b>        | <b>82.00%</b>        |
| ClarifyCoder (w/ DS Coder,<br>loss=answer only) | <b><u>63.61%</u></b> | <b><u>51.93%</u></b> | <b><u>74.85%</u></b> | <b><u>83.29%</u></b> |

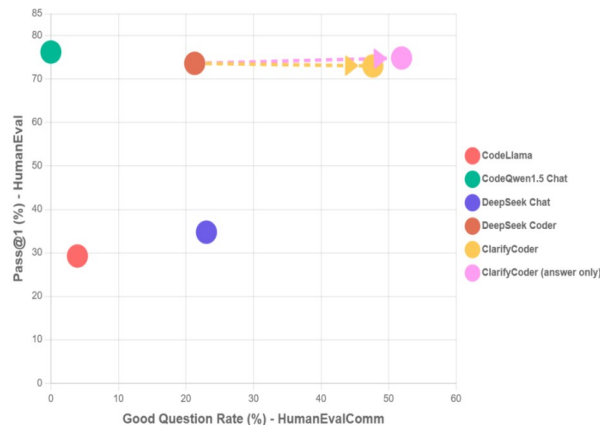
Table 3. Results on *HumanEvalComm* and *HumanEval* benchmarks for different LLMs. **First-place** and **Second-place** results are underlined and bolded, while **top 4** results are bolded. "loss=answer only" denotes the loss during fine-tuning, calculated only on the answers rather than on both the problems and the answers.

# Performance Analysis (2 / 2)



● CodeLlama ● CodeQwen1.5 Chat ● DeepSeek Chat ● DeepSeek Coder  
● ClarifyCoder ● ClarifyCoder (answer only)

(a) Multi-dimensional performance profile for clear (HumanEval) and ambiguous requirements (HumanEvalComm).



(b) a scatter plot of the trade-off between clarification-seeking ability and code generation performance. Dashed arrow represents the impact of Clarification-aware fine-tuning on the original DeepSeek Coder model.

Fig. 3. Comprehensive performance analysis of ClarifyCoder and baseline LLMs. "loss=answer only" denotes the loss during fine-tuning, calculated only on the answers rather than on both the problems and the answers.

# Ratio $r$ of standard and synthetic data.

- We changed the ratio  $r$  from 20% to 100%, with downsampling.
- Even with a modest 20% ratio, utilizing just 2,191 clarify-aware samples, we observed a notable improvement in clarify-awareness performance

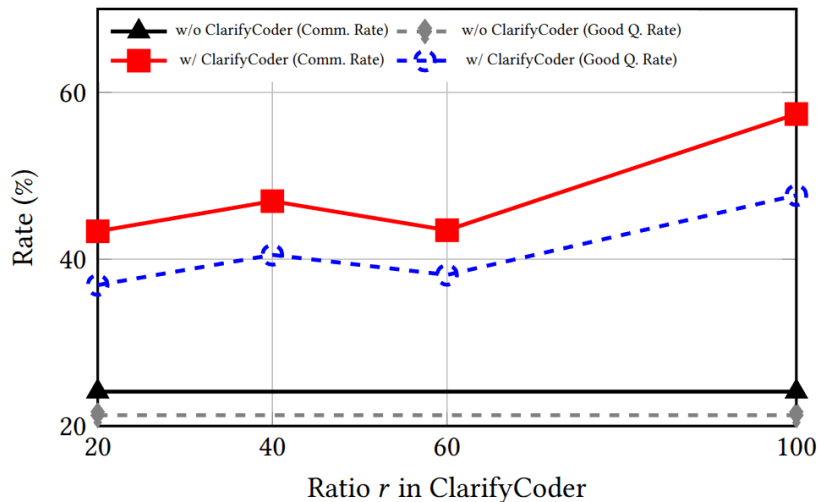


Fig. 6. ClarifyCoder performance with varying ratio  $r$ .

# Comparison with Training-Free Methods

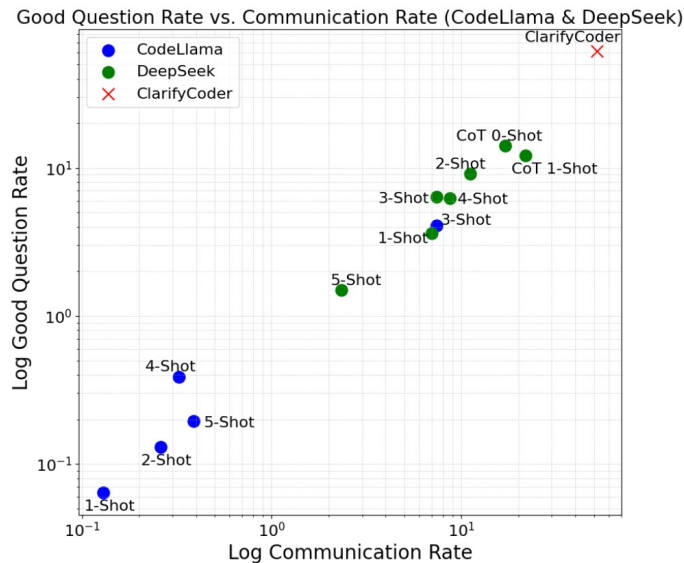


Fig. 4. Results on *HumanEvalComm V2* For In-Context Learning and Chain-of-Thought for CodeLlama and DeepSeek. The reported Communication rates and Good Question Rates are based on a log scale.

# Compare with other LLM agents

Results of different methods on HumanEval benchmark (well-specified coding tasks). Pass@1 and Test PR measure code generated in the first (only) round. The top two results are bolded. Pass rates are considered unavailable for methods where user interaction to perform clarification is enforced.

| Method                                                  | Method Type  | # LLM calls | Interaction Enforced | Pass@1        | Test Pass Rate |
|---------------------------------------------------------|--------------|-------------|----------------------|---------------|----------------|
| TICODER                                                 | LLM Workflow | 2           | Yes                  | N/A           | N/A            |
| ClarifyGPT                                              | LLM Workflow | 4           | Yes                  | N/A           | N/A            |
| ClariGen                                                | LLM Workflow | 3           | Yes                  | N/A           | N/A            |
| Okanagan <sub>(Base=ChatGPT)</sub>                      | LLM Workflow | 3           | No                   | 27.45%        | 33.45%         |
| Okanagan <sub>(Base=DS Coder)</sub>                     | LLM Workflow | 3           | No                   | 21.25%        | 24.3%          |
| DeepSeek Chat                                           | Single LLM   | 1           | No                   | 34.76%        | 53.00%         |
| DeepSeek Coder                                          | Single LLM   | 1           | No                   | <b>73.62%</b> | <b>82.36%</b>  |
| ClarifyCoder <sub>(w/ DS Coder)</sub>                   | Single LLM   | 1           | No                   | 73.01%        | 82.00%         |
| ClarifyCoder <sub>(w/ DS Coder, loss=answer only)</sub> | Single LLM   | 1           | No                   | <b>74.85%</b> | <b>83.29%</b>  |

# Pretraining & fine-tuning in code

Pretraining:

[https://github.com/rasbt/LLMs-from-scratch/blob/main/ch05/01\\_main-chapter-code/ch05.ipynb](https://github.com/rasbt/LLMs-from-scratch/blob/main/ch05/01_main-chapter-code/ch05.ipynb)

[https://github.com/rasbt/LLMs-from-scratch/blob/main/ch06/01\\_main-chapter-code/ch06.ipynb](https://github.com/rasbt/LLMs-from-scratch/blob/main/ch06/01_main-chapter-code/ch06.ipynb)

# NanoGPT in notebook

GPT (Zero to Hero), by AK:

[https://colab.research.google.com/drive/1JMLa53HDuA-i7ZBmqV7ZnA3c\\_fvtXnx-](https://colab.research.google.com/drive/1JMLa53HDuA-i7ZBmqV7ZnA3c_fvtXnx-)

(Strongly recommend to watch the video and the code)

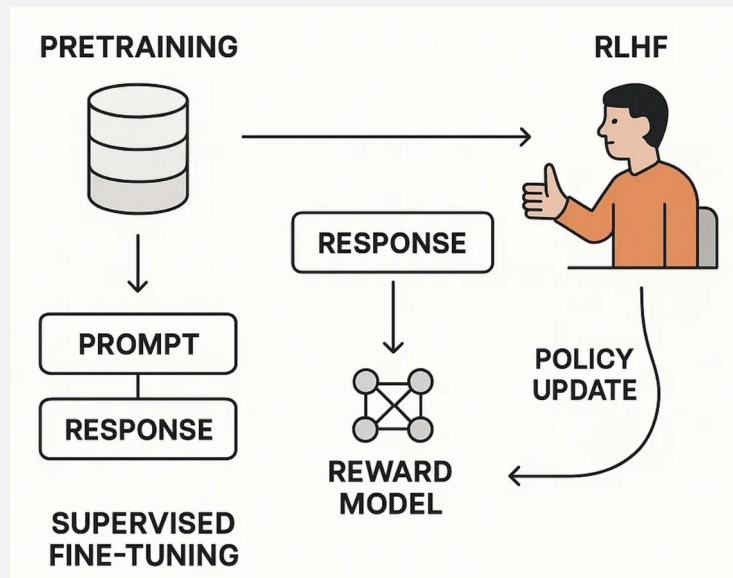
# RLHF (Reinforcement Learning from Human Feedback)

RLHF is a way to train an AI model so that its outputs better match what humans prefer—not just what is statistically likely in the training data.

A useful way to think about the whole pipeline is: **Pretraining** → **SFT** → **RLHF**

During **pretraining**, the model mainly learns to predict the next token. During **SFT (supervised fine-tuning)**, humans provide examples of good responses and the model learns to imitate them. RLHF then goes one step further: instead of only showing the model a correct answer, humans compare multiple model answers and indicate which one is better.

**SFT** learns from the correct answer directly. **RL** learns from a reward signal.



# RLHF

|                 | SFT                             | RL                                        |
|-----------------|---------------------------------|-------------------------------------------|
| Full name       | Supervised Fine-Tuning          | Reinforcement Learning                    |
| Training signal | Target answer                   | Reward                                    |
| Data looks like | (prompt, good response)         | (prompt, generated response, reward)      |
| Model asks      | “What answer should I imitate?” | “What kind of answer gets higher reward?” |
| Objective       | Match reference tokens          | Maximize expected reward                  |
| Typical loss    | Cross-entropy                   | Policy optimization objective             |

# RLHF

Step 1 performs instruction fine-tuning on 13k training examples

- This aligns the model behavior with what we would expect of a chat agent
- But the diversity of the interactions might still be limited by the contents of the training data
- Rather, preference signals are a more expressive supervision surface (easier for humans to label their preferred response than to generate one to SFT!)

Step 1

**Collect demonstration data, and train a supervised policy.**

A prompt is sampled from our prompt dataset.

Explain the moon landing to a 6 year old

A labeler demonstrates the desired output behavior.

Some people went to the moon...

This data is used to fine-tune GPT-3 with supervised learning.

SFT

Step 2

**Collect comparison data, and train a reward model.**

A prompt and several model outputs are sampled.

Explain the moon landing to a 6 year old

A B  
Explain gravity... Explain war...  
C D  
Moon is natural satellite of... People went to the moon...

A labeler ranks the outputs from best to worst.

D > C > A = B

This data is used to train our reward model.

RM

Step 3

**Optimize a policy against the reward model using reinforcement learning.**

A new prompt is sampled from the dataset.

Write a story about frogs

The policy generates an output.

PPO

Once upon a time...

The reward model calculates a reward for the output.

RM

The reward is used to update the policy using PPO.

$r_k$

Figure 2: A diagram illustrating the three steps of our method: (1) supervised fine-tuning (SFT), (2) reward model (RM) training, and (3) reinforcement learning via proximal policy optimization (PPO) on this reward model. Blue arrows indicate that this data is used to train one of our models. In Step 2, boxes A-D are samples from our models that get ranked by labelers. See Section 3 for more details on our method.

2022 (OpenAI) (Arxiv) [InstructGPT] [RLHF] Training language models to follow instructions with human feedback



# RLHF

- In Step 2, takes 33k prompts and samples a collection of responses from the instruction fine-tuned model for each one
- The human labeler ranks the  $K \in \{4, \dots, 9\}$  responses
- The reward model is a copy of the Step-1 LLM, but with the softmax over words replaced so that it outputs a single scalar value, i.e. the reward
- The model is trained so that rewards of the higher ranking (winning) responses are larger than those of the lower ranking (losing) responses

Step 2

**Collect comparison data,  
and train a reward model.**

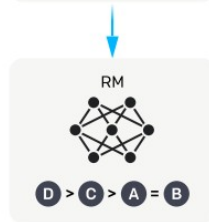
A prompt and  
several model  
outputs are  
sampled.



A labeler ranks  
the outputs from  
best to worst.



This data is used  
to train our  
reward model.



# RLHF- Step 3

PPO does not directly use human preference pairs.  
It typically uses a **reward signal** produced by the Reward Model.

Prompt



LLM generates response



Reward Model



Reward score



PPO



Update LLM

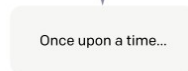
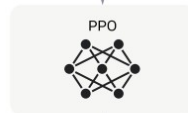
Step 3

**Optimize a policy against the reward model using reinforcement learning.**

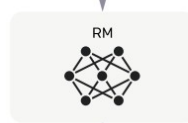
A new prompt is sampled from the dataset.



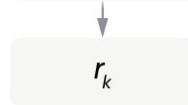
The policy generates an output.



The reward model calculates a reward for the output.

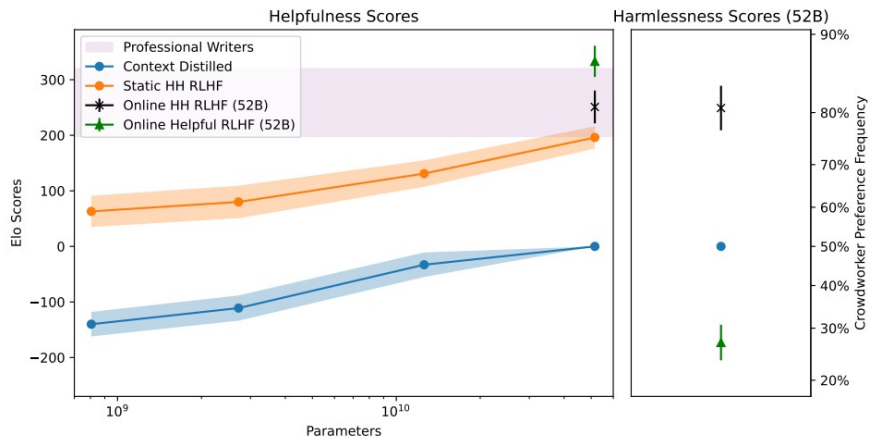


The reward is used to update the policy using PPO.



# RLHF results

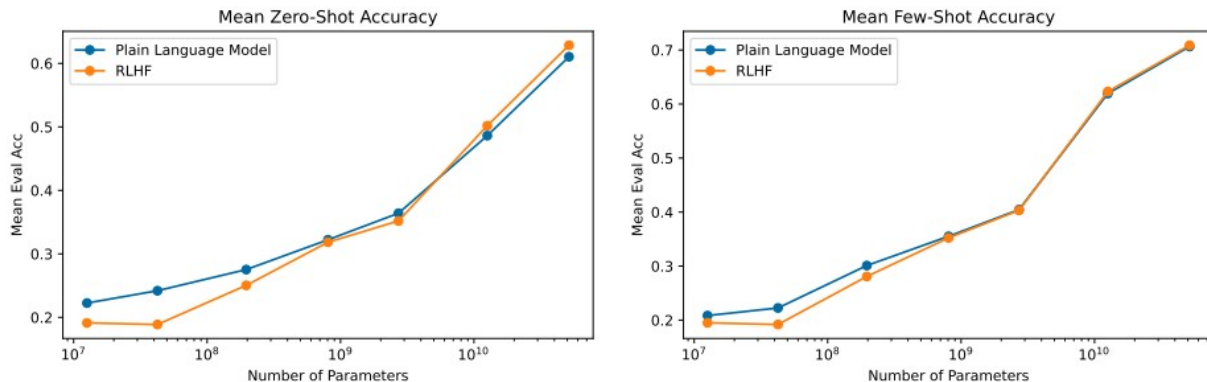
## RLHF Increases helpfulness



**Figure 1** This plot summarizes crowdworker preferences for a variety of models, including context-distilled models, RLHF models trained on our 'static' dataset, and RLHF models trained by an iterated 'online' method for either helpfulness and harmlessness (HH) or for helpfulness only. We present both Elo scores and a match to the frequency with which crowdworkers prefer samples as compared to the 52B context-distilled model. For both helpfulness and harmlessness, a higher score is more desirable.

# RLHF results

Zero shot and few shot is flat



**Figure 3** RLHF model performance on zero-shot and few-shot NLP tasks. For each model size, we plot the mean accuracy on MMMLU, Lambada, HellaSwag, OpenBookQA, ARC-Easy, ARC-Challenge, and TriviaQA. On zero-shot tasks, RLHF training for helpfulness and harmlessness hurts performance for small models, but actually improves performance for larger models. Full results for each task are given in Figure 28 (zero-shot) and Figure 29 (few-shot).

# RLHF code tutorial

<https://blog.ml.cmu.edu/2025/06/01/rhlf-101-a-technical-tutorial-on-reinforcement-learning-from-human-feedback/>

# Reinforcement Learning (RL)

“The goal of Reinforcement Learning (RL) is to learn a good strategy for the agent from experimental trials and relative simple feedback received. With the optimal strategy, the agent is capable to actively adapt to the environment to maximize

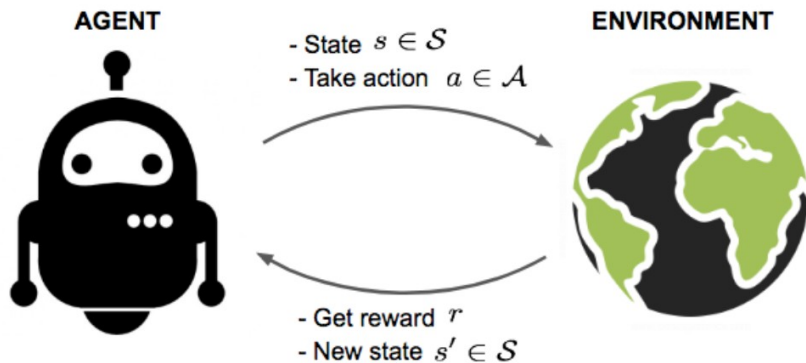


Figure 1: An agent interacts with the environment, trying to take smart actions to maximize cumulative rewards.

SFT: "Here is the correct answer. Learn to imitate it."

RL: "Try an answer. I will tell you how good it is."

<https://lilianweng.github.io/posts/2018-02-19-rl-overview/>

# Proximal Policy Optimization (PPO)

Problem:

If we update the model too aggressively based on reward, the policy may become unstable.

$$\underset{\theta}{\text{maximize}} \quad \hat{\mathbb{E}}_t \left[ \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \hat{A}_t \right] - \beta \hat{\mathbb{E}}_t [\text{KL}[\pi_{\theta_{\text{old}}}(\cdot | s_t), \pi_{\theta}(\cdot | s_t)]]$$

PPO idea:

Increase expected reward while limiting how much the policy changes at each step.

How much did the probability of this action change compared with the old model?

# DPO: Direct Preference Optimization

If we already know that response A is preferred over B, can we directly train the model from that preference?

DPO directly optimizes preference without explicitly training a Reward Model or running PPO.

Motivation:

RLHF is relatively complex:

Preference



Reward Model



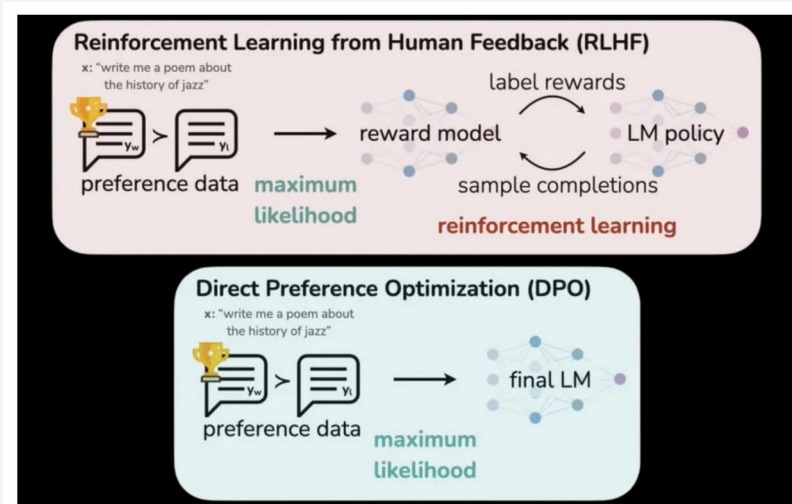
Reward



PPO



LLM



<https://thisisrishi.medium.com/direct-preference-optimization-dpo-in-llms-21225b991f4e>

# Summary

Code of DPO/PPO:

<https://medium.com/@ufuk.birbiri/reinforcement-learning-from-human-feedback-using-ppo-ec9157baa41d>

| Method | Training signal     | Core idea                                        |
|--------|---------------------|--------------------------------------------------|
| SFT    | Correct response    | Imitate good answers                             |
| RL     | Reward              | Maximize reward                                  |
| PPO    | Reward              | Maximize reward without changing policy too much |
| DPO    | Chosen vs. rejected | Directly learn which response is preferred       |

# Reasoning ability

## LLM Reasoning Definition:

The capability of an LLM to perform intermediate computation and inference in order to derive an answer that is not directly available from the input or memorized knowledge.

**Examples:** math, logic, **coding**, planning, scientific reasoning.

## Key idea:

Problem → Intermediate inference → Conclusion

**Important limitation:** fluent reasoning  $\neq$  correct reasoning.

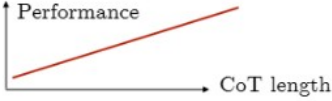
# Reasoning model

Reasoning model is relied on Chain-of-Thought Reasoning trace to solve more complex problems in coding, math, ect.

- 1) Test time scaling
- 2) Train time scaling

Reasoning Models

{ Train-time scaling: RLVR, GRPO, reasoning SFT  
Test-time scaling: longer CoT, search, sampling, verification

|                    | Description                                                                                                | Illustration                                                                       |
|--------------------|------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| Train-time scaling | Run RL for longer to let the model learn how to produce CoT-style reasoning traces before giving an answer |  |
| Test-time scaling  | Let the model think longer before providing an answer with budget forcing keywords such as "Wait"          |  |

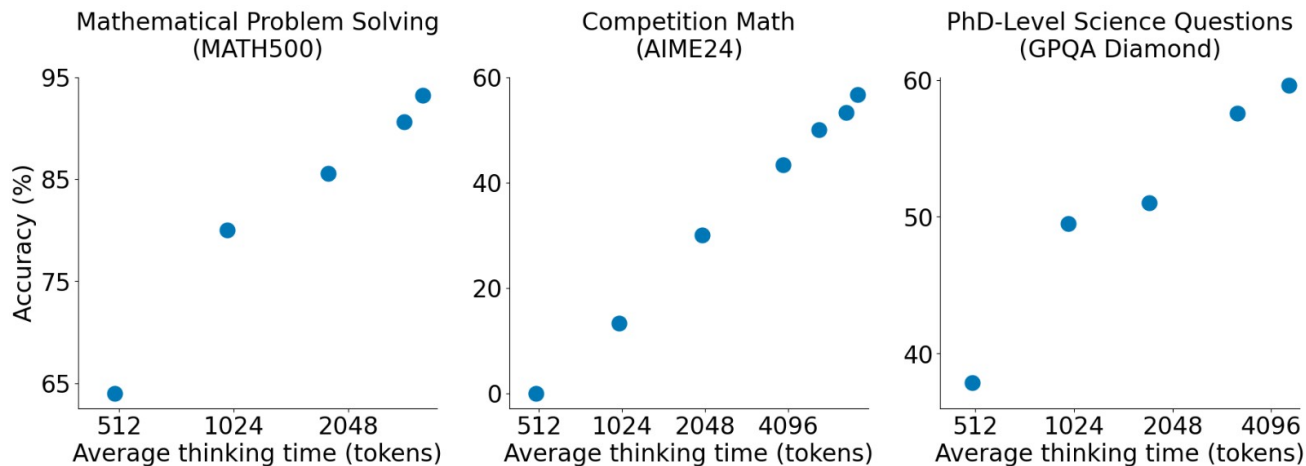
<https://github.com/afshinea/stanford-cme-295-transformers-large-language-models/blob/main/en/cheatsheet-transformers-large-language-models.pdf>



# s1: Simple test-time scaling

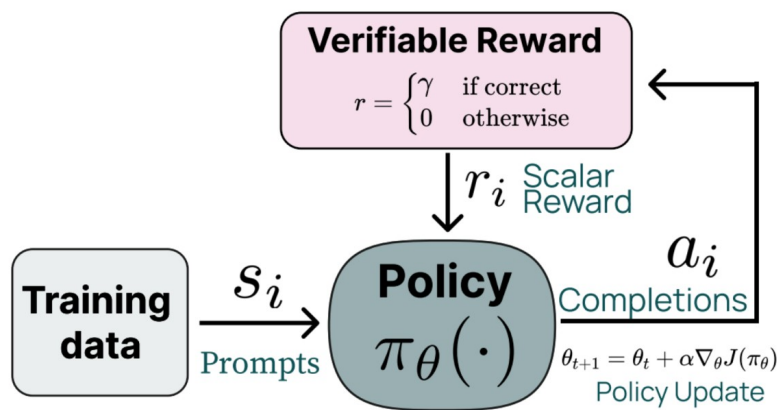
<https://simplescaling.github.io/>

We introduce s1: a minimal recipe for test-time scaling and strong reasoning performance matching o1-preview with just 1,000 examples & budget forcing



# Train time scaling- RLVR

RLVR + GRPO are techniques for training reasoning models.



**Figure 18** An overview of how Reinforcement Learning with Verifiable Rewards (RLVR) works. We sample completions from a policy model given a set of prompts, and verify their correctness using a deterministic function. If the answer is verifiably correct, we provide reward of  $\alpha$ , otherwise 0. We then train against this reward using PPO.

# Train time scaling- GRPO

RL with GRPO. The training of DeepSeek-R1-Zero begins with the DeepSeek-v3 [2] base model. We directly finetune this base model via RL. In particular, authors in [1] select

Group Relative Policy Optimization (GRPO) [3], which is depicted in the figure above, as their RL algorithm. The selection of RL algorithms for LLM training is an open and active research topic. Traditionally, researchers have used PPO for training LLMs, but there is a recent trend towards adopting simpler RL algorithms—such as REINFORCE or GRPO—for LLM training. The main reasons provided for the selection of GRPO in [1] are:

- A reduction in the cost of RL training.
- The elimination of the critic model, which is (usually) the same size as the policy model (i.e. the LLM itself).

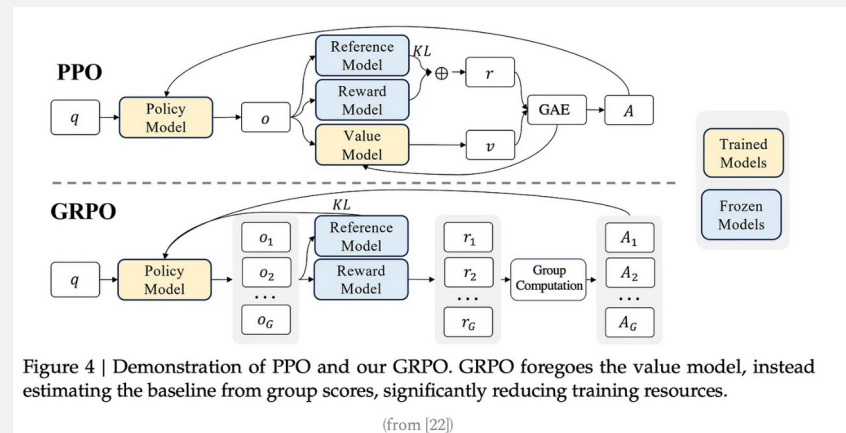


Figure 4 | Demonstration of PPO and our GRPO. GRPO foregoes the value model, instead estimating the baseline from group scores, significantly reducing training resources.

(from [22])

<https://cameronrwolfe.substack.com/p/demystifying-reasoning-models?open=false#footnote-anchor-13>

# Reasoning model case study: coding

Evaluating Large Language Models Trained on Code  
GitHub, Jul 2022

Main contributions:

- Introduces **Codex**, a GPT-style model trained/fine-tuned on public GitHub code.
- Studies whether LLMs can generate **functionally correct Python programs from natural-language descriptions**.
- Introduces the **HumanEval** benchmark.
- Introduces/popularizes **pass@k** as a functional-correctness metric for code generation.
- Shows that generating multiple candidate solutions can dramatically improve success rates. [DOI](#)

Conceptual pipeline:

Natural-language specification → LLM → Generated code → Unit tests

---

## Evaluating Large Language Models Trained on Code

---

Mark Chen<sup>\*1</sup> Jerry Tworek<sup>\*1</sup> Heewoo Jun<sup>\*1</sup> Qiming Yuan<sup>\*1</sup> Henrique Ponde de Oliveira Pinto<sup>\*1</sup>  
Jared Kaplan<sup>\*2</sup> Harri Edwards<sup>1</sup> Yuri Burda<sup>1</sup> Nicholas Joseph<sup>2</sup> Greg Brockman<sup>1</sup> Alex Ray<sup>1</sup> Raul Puri<sup>1</sup>  
Gretchen Krueger<sup>1</sup> Michael Petrov<sup>1</sup> Heidy Khlaaf<sup>3</sup> Girish Sastry<sup>1</sup> Pamela Mishkin<sup>1</sup> Brooke Chan<sup>1</sup>  
Scott Gray<sup>1</sup> Nick Ryder<sup>1</sup> Mikhail Pavlov<sup>1</sup> Alethea Power<sup>1</sup> Lukasz Kaiser<sup>1</sup> Mohammad Bavarian<sup>1</sup>  
Clemens Winter<sup>1</sup> Philippe Tillet<sup>1</sup> Felipe Petroski Such<sup>1</sup> Dave Cummings<sup>1</sup> Matthias Plappert<sup>1</sup>  
Fotios Chantzis<sup>1</sup> Elizabeth Barnes<sup>1</sup> Ariel Herbert-Voss<sup>1</sup> William Hebgren Guss<sup>1</sup> Alex Nichol<sup>1</sup> Alex Paino<sup>1</sup>  
Nikolas Tezak<sup>1</sup> Jie Tang<sup>1</sup> Igor Babuschkin<sup>1</sup> Suchir Balaji<sup>1</sup> Shantanu Jain<sup>1</sup> William Saunders<sup>1</sup>  
Christopher Hesse<sup>1</sup> Andrew N. Carr<sup>1</sup> Jan Leike<sup>1</sup> Josh Achiam<sup>1</sup> Vedant Misra<sup>1</sup> Evan Morikawa<sup>1</sup>  
Alec Radford<sup>1</sup> Matthew Knight<sup>1</sup> Miles Brundage<sup>1</sup> Mira Murati<sup>1</sup> Katie Mayer<sup>1</sup> Peter Welinder<sup>1</sup>  
Bob McGrew<sup>1</sup> Dario Amodei<sup>2</sup> Sam McCandlish<sup>2</sup> Ilya Sutskever<sup>1</sup> Wojciech Zaremba<sup>1</sup>

# Codex: Training LLMs on Code

Training data:

- Public GitHub repositories
- Initial collection: about **54 million repositories**
- About **179 GB of unique Python files**
- After filtering: about **159 GB** of Python code.

Language modeling + large-scale source code→code generation capability

This paper helped establish that scaling language models on source code can produce surprisingly strong program-synthesis capabilities.

# HumanEval: How Do We Evaluate Generated Code?

The paper introduces **HumanEval**:

- **164 handwritten Python programming problems**
- Each problem contains:
  - function signature
  - docstring / natural-language specification
  - hidden unit tests
- Model generates the function implementation.
- A solution is correct if it **passes the tests**.

[https://huggingface.co/datasets/openai/openai\\_humaneval](https://huggingface.co/datasets/openai/openai_humaneval)

# Pass@k

Metric for problems where the model can generate k attempts; counts success if any of the k attempts is correct. Pass@k is common in coding and tool-using tasks. (<https://www.braintrust.dev/encyclopedia/pass-at-k>)

Takeaways:

Code generation is not necessarily single-shot generation

Sampling multiple candidates can be considered a form of inference-time scaling.

# Results

The model often **has the ability to generate a correct solution**, even when the most likely single generation is incorrect.

This foreshadows later work on:

- sampling
- self-consistency
- search
- reranking
- test-time / inference-time compute.

<https://arxiv.org/pdf/2107.03374>

*Table 1.* Codex, GPT-Neo, & TabNine evaluations for HumanEval  
We find that GPT-J pass@1 is between Codex-85M and Codex-300M performance.

|              | PASS@ <i>k</i> |               |                |
|--------------|----------------|---------------|----------------|
|              | <i>k</i> = 1   | <i>k</i> = 10 | <i>k</i> = 100 |
| GPT-NEO 125M | 0.75%          | 1.88%         | 2.97%          |
| GPT-NEO 1.3B | 4.79%          | 7.47%         | 16.30%         |
| GPT-NEO 2.7B | 6.41%          | 11.27%        | 21.37%         |
| GPT-J 6B     | 11.62%         | 15.74%        | 27.74%         |
| TABNINE      | 2.58%          | 4.35%         | 7.59%          |
| CODEX-12M    | 2.00%          | 3.62%         | 8.58%          |
| CODEX-25M    | 3.21%          | 7.1%          | 12.89%         |
| CODEX-42M    | 5.06%          | 8.8%          | 15.55%         |
| CODEX-85M    | 8.22%          | 12.81%        | 22.4%          |
| CODEX-300M   | 13.17%         | 20.37%        | 36.27%         |
| CODEX-679M   | 16.22%         | 25.7%         | 40.95%         |
| CODEX-2.5B   | 21.36%         | 35.42%        | 59.5%          |
| CODEX-12B    | 28.81%         | 46.81%        | 72.31%         |

# Limitations

## Reasoning limitations

Codex struggles when the specification requires:

- long sequences of operations
- correctly binding many operations to different variables

Performance declines substantially as problems require longer compositional chains.

## Correctness limitations

Generated code can:

- look plausible but be wrong
- reference undefined variables/functions
- contain semantic or security problems